# DistillToMCU: Distilling Cloud-LLM Control Behavior into Confidence-Calibrated Rules for LLM-Free Microcontroller Execution

Yuxiang Zhang[a,*]

[a] *Anhui Normal University, Huajin Campus, No. 189 Jiuhua South Road, Yijiang District, Wuhu, 241002, Anhui, China*

**Abstract**

Large language models (LLMs) increasingly drive the control loop of embedded devices, but keeping the model in the loop on every decision imposes network latency, cost, privacy exposure, and offline fragility that microcontrollers cannot absorb. We present DistillToMCU, a system that observes a cloud LLM's structured tool-call decisions and distills them into confidence-calibrated interval rules executed locally on an ESP32-S3 with no LLM at runtime. Distillation (COMIC) combines online quantile estimation, variance tracking, split-conformal calibration, minimum-description-length consolidation, and harmful-condition pruning into rules with per-rule confidence and a five-state lifecycle. Rule selection uses perturbed-mean selection (PMS), a follow-the-perturbed-leader scheme with 2-byte-per-rule uint8 counters. In evaluations with more than 19,000 real cloud calls across four synthetic and five real datasets (smart homes, industrial energy, urban air quality), a held-out protocol, and teacher replay, DistillToMCU reaches 85.4% held-out autonomy over 36 blocks, statistically on par with hand-written, one-shot, and ESP-Claw-style rules, with fidelity between 57% and 89% of the teacher's self-agreement baseline on synthetic data. On UCI, where full-horizon real-sensor autonomy is data-capped (bootstrap 95% CI 18.2–46.5%), its warm-up rules keep 50.0% precision versus

*Corresponding author. Tel.: +86 19397619346; E-mail address: yuxiang.zhang@ahnu.edu.cn

16.5% for a batch decision tree on a 1,440-snapshot replay, showing calibrated abstention beats aggressive imitation on low-action real data; the same advantage holds on industrial energy (52.8% versus 30.0%). Replacing the teacher shifts autonomy by only 1.7–3.7 points, and on-board rule matching completes in 1.48 ms (p50).

*Keywords:* LLM agents, behavior distillation, microcontrollers, rule mining, conformal prediction, multi-armed bandits

---

## 1. Introduction

Language models have moved from chatbots into the physical world. Assistants now decide when to switch a light, start a fan, or open a valve, and a growing line of systems wires an LLM into the control path of embedded devices, with smart homes as the most studied instantiation [1], [2], [3]. The same question reaches every microcontroller-class device that must act on physical state — industrial panels, mobile robots, wearables, and medical monitors — where a cloud round trip is too slow, too exposed, or simply unavailable. The attraction is real: natural language replaces rule editors, and the model absorbs knowledge about comfort, efficiency, and safety that hand-written automations do not capture. The cost is equally real. A cloud decision takes a mean of about 4.8 s and a median of 3.9 s in our measurements[1], transmits the device's sensor state to a third party, stops working without connectivity, and bills every interaction. Microcontrollers, the class of device that actually drives relays, LEDs, and valves in this setting, typically expose a few hundred kilobytes of RAM. They cannot host the model, and they cannot hide a roughly 4.8 s round trip inside an interactive control loop.

[1]Cloud round-trip latency computed from the latency_ms field of logged llm_response records in the online held-out and same-data cross-teacher run traces: 6,226 held-out calls (mean 4.27 s, median 3.82 s) plus 806 cross-teacher calls, pooled n = 7,032 (mean 4.81 s, median 3.92 s); an independent scripted cross-model pairing of 322 calls gives mean 4.82 s, median 4.20 s.

Two design lines dominate the 2026 literature, and both keep the LLM present. The first line keeps the model in the loop at the edge. HearthNet deploys a set of persistent, role-specialized LLM agents at the home hub and uses hosted LLM inference for planning and conflict resolution [1]. EdgeTalk-MCU couples a local small language model with a safety shield on MCU-class hardware [2]. ESP-Claw executes Lua-generated automations under continuous LLM supervision [3], and DCP adds protocol-level protection around the LLM bridge [4]. The second line compresses past behavior for agents that still run an LLM. ClawTrace records cost-attributed traces and writes skill patches for cloud agents [5], RIMRULE distills minimum-description-length rules from failure traces and injects them back into prompts [6], and Skill-DisCo compiles successful trajectories into reusable procedural subgraphs [7]. Related work on plan reuse and skill packaging follows the same pattern [8], [9], [10]. Neither line answers the question this paper asks: can the distilled behavior run on the microcontroller itself, with the LLM absent at runtime?

The Experience Compression Spectrum gives this question a name. It organizes agent memory, skills, and rules into four levels of increasing compression and observes that the declarative-rule level, its Level 3, remains largely empty [11]. The open-source WireClaw project demonstrates the appetite for the idea, executing chat-configured rules offline on an ESP32 [12], but it configures rules in one conversation, never learns from observed decisions, assigns no confidence, and is not peer-reviewed. We therefore target the peer-reviewed gap: an automated, incremental, confidence-calibrated distillation of cloud LLM control behavior into rules an MCU can execute without the LLM.

This paper makes one core contribution and two supporting ones. The core contribution is COMIC, a distillation pipeline that turns the stream of structured tool-call decisions into interval rules. It chains online quantile estimation (CKMS) and online variance tracking (Welford) with background distribution filtering, split-conformal calibration, minimum-description-length consolidation, and a leave-one-out pruning step that removes conditions whose removal does not hurt coverage. Every rule carries a Wilson-score confidence and advances

through a five-state lifecycle driven by evidence and freshness. The first supporting contribution is perturbed-mean selection (PMS), a follow-the-perturbed-leader selector that stores uint8 alpha/beta counters, two bytes per rule, and resolves rule conflicts under drift. The second supporting contribution is a trace-driven hardware-in-the-loop implementation on an ESP32-S3 with state-aware idempotent actuation.

The evaluation is designed around the question of fairness. All eight compared methods share one held-out protocol: warm-up on days 1 to 21, evaluation on days 22 to 30. DistillToMCU reaches 85.4% held-out autonomy across 36 blocks, statistically indistinguishable from hand-written rules, one-shot rules, and ESP-Claw-style rules, while a decision tree trained on the same window reaches 99.4% and an online daily-refit tree 99.7%. That ranking flips once decisions are checked against the teacher. On UCI real sensor data, the tree acts on nearly every snapshot with 16.5% precision on the extended 1,440-snapshot replay, while DistillToMCU abstains and keeps 50.0% precision. Fidelity, measured against the teacher's own 80.0% to 96.7% self-agreement baseline, reaches 57% to 89% of that baseline on synthetic data. Replacing the teacher model moves autonomy by only 1.7 to 3.7 percentage points on identical sensor sequences, and rule matching on the device completes in 1.48 ms (p50). Section 2 positions these results against prior work; Section 3 describes the system, Sections 4 and 5 cover setup and results, and Sections 6 and 7 discuss and bound the claims.

## 2. Related Work

### *2.1. LLM-in-the-loop edge control*

Keeping the model inside the control loop is the current default. HearthNet orchestrates a small set of persistent LLM agents at the home hub over MQTT and Git-backed shared state, separating planning, verification, authorization, and actuation while hosted inference supplies the reasoning [1]. EdgeTalk-MCU places a local model plus a shield beside the MCU so that interpretation and

vetting happen on-device [2]. ESP-Claw gives the model a Lua scripting engine and keeps it supervising execution [3], and DCP hardens the bridge between the device and the model [4]. CIDER instead moves intent reasoning into an on-device small model with an ontology [13]. All of these designs assume the model, large or small, remains available at decision time. DistillToMCU removes that assumption: the model appears only as a teacher during an observation phase, and the rules it leaves behind are the runtime system.

### *2.2. Trace distillation for LLM agents*

Distilling behavior from trajectories has become an active line of research, but its outputs target agents that still run an LLM. ClawTrace attaches per-step cost to trajectories and emits preserve, prune, and repair patches that a cloud agent applies in later sessions [5]. RIMRULE mines rules from failure traces, consolidates them with a minimum-description-length objective, and injects them into prompts at inference time, so the model keeps running and keeps reading the rules [6]. Skill-DisCo compiles successful trajectories in finite-state settings into parameterized control-flow subgraphs that agents invoke [7]. AgentReuse reuses generated plans for similar requests to cut latency [8], Skill-as-Pseudocode rewrites skill libraries into typed pseudocode [9], and SkCC compiles skills for portability and security across agent frameworks [10], and Skill-SD turns trajectory summaries into training supervision for the student model [14]. These systems cut reasoning cost. None removes the reasoner. DistillToMCU applies the same intuition—treating behavior as a first-class artifact—but compiles it to a symbolic form a commodity microcontroller can evaluate locally.

### *2.3. Rule mining and resource-constrained selection*

Learning rules from sensor traces predates the LLM era. HomeSGN pairs a generator and a scorer to mine home automations [15], and Hoffner et al. derive recommendation rules from observed behavior [16]. These systems mine the sensor stream. Ours mines the controller's behavior, the signal that encodes intent. The selection layer draws on the bandit literature. Thompson sampling gives

near-optimal regret bounds [17], yet its exact posterior sampling is too expensive for microcontroller-class hardware, and ordinal approximations on FPGA SoCs [18] plus communication-quantized and minimal variants [19], [20] pursue the same trade-off. Our selector belongs to the follow-the-perturbed-leader family [21]: it perturbs the empirical mean by uniform noise instead of sampling a Beta posterior, costs two bytes per rule, and is evaluated under stationary and drifting environments. Calibration draws on split conformal prediction [22], [23], online quantile summaries [24], and online variance estimation [25]; rule consolidation follows the minimum description length principle [26], and per-rule confidence uses the Wilson score interval [27].

### *2.4. Positioning*

The closest conceptual neighbors are WireClaw, which executes LLM-generated rules offline on an ESP32 but with one-shot configuration, no trace distillation, no confidence calibration, and no lifecycle [12], and the Experience Compression Spectrum, which names the declarative-rule level without instantiating it [11]. DistillToMCU is, to our knowledge, the first peer-reviewed system that distills cloud-LLM control behavior incrementally into calibrated MCU-executable rules and validates the result in hardware. Recent neuro-symbolic agent frameworks follow the same distillation intuition with different outputs: WALL-E 2.0 extracts symbolic action rules and knowledge graphs from exploration trajectories to align an LLM's world model but keeps the LLM as the planner [37], and AgentDistill reuses teacher-generated MCP modules in a smaller student agent that still runs an LLM [38]. Both compile behavior into artifacts an LLM consumes; COMIC compiles behavior into rules an MCU executes with no model present. Three properties together separate it from an incremental interval learner on the same hardware: per-rule conformal calibration makes every coverage claim auditable, a five-state lifecycle retires stale rules instead of letting them accumulate, and the hardware-in-the-loop validation replays released traces that readers can rerun. The closest scripted counterpart, ESP-Claw, runs an LLM-supervised engine on the same class of hardware, but it assigns no per-

rule confidence, has no lifecycle, and is not validated with released traces.

## 3. System Design

### *3.1. Overview and assumptions*

The system spans two places and five layers. On the device, a sensor layer captures readings, a trace layer records every decision, a rule layer matches and maintains rules, and an execution layer drives actuators. On a PC or gateway, a distillation layer reads the trace log, learns new rules, and pushes them back. The device performs only three duties: sample sensors, match rules, and drive outputs. Three assumptions bound the design. First, the teacher LLM expresses every control decision as a structured tool call with a device name, a command, and optional parameters, so the trace is machine-readable without parsing free text. Second, control behavior is approximately interval-shaped: the teacher turns a fan on within some temperature range and keeps a light off within some light range, and deviations are noise rather than adversarial examples. Third, behavior drifts slowly, which motivates freshness and a lifecycle but does not require instantaneous adaptation. None of these assumptions requires the teacher to be correct; the system reproduces the teacher, and correctness is inherited, not created. Figure 1 summarizes the layered design and the split between the device and the PC/gateway.

### *3.2. COMIC: calibrated interval distillation*

COMIC converts cloud decisions into rules in six steps. Step one groups positive samples by the observed (device, command) pair. Step two estimates per-sensor value distributions online. A CKMS quantile summary [24] tracks the empirical distribution of each sensor value inside each action group with a target approximation error of 0.01, and a Welford accumulator [25] tracks mean and variance, so both scale in constant memory across a 30-day stream. Step three forms interval candidates, taking the quantile envelope and expanding it by a data-driven margin, then clips the result against physical plausibility

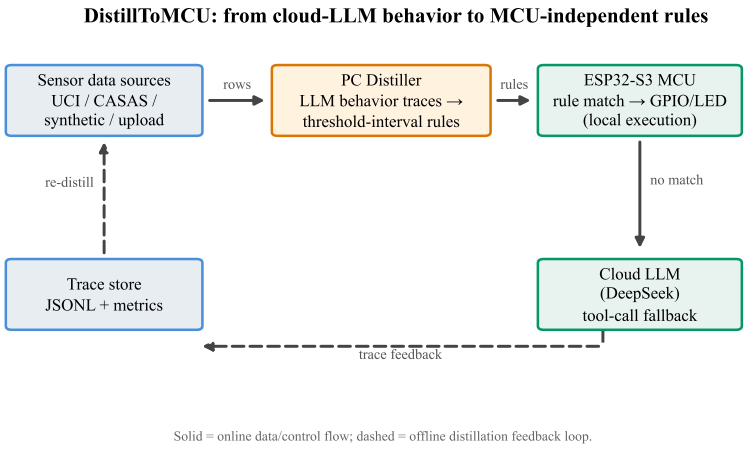



Figure 1: Five-layer architecture: interaction, trace, distillation, lifecycle, and execution, split between the ESP32-S3 and a PC/gateway distiller.

ranges per sensor. Step four filters candidates for discriminative power. A condition survives only if the value distribution it carves out differs from the background distribution of the same sensor across all other actions, which removes the vacuous conditions (for example a light range that covers the entire plausible band) that would otherwise make every rule match everything. Step five calibrates each rule with split conformal prediction [22], [23]: calibration is computed on a held-out fraction of the stream so the reported coverage is not inflated by the samples that built the interval, with a nominal 85% coverage target widened adaptively when the calibration set is small. Step six consolidates candidates under a minimum description length objective [26] and runs a leave-one-out pruning pass that deletes any condition whose removal does not reduce coverage on the evidence set. Each surviving rule stores its conditions, the majority-voted parameters, a Wilson-score confidence interval [27], an evidence count, and a freshness timestamp.

The pipeline itself, rather than its parameter settings, yields two properties. The interval envelope generalizes to states the teacher has not seen: a rule learned from temperatures near 28 °C and 31 °C fires at 29.5 °C, which an

exact cache cannot do. The conformal step and the pruning step make the rule's coverage claim auditable, so the executor can weigh a match by how often that rule was right, not merely by whether it fired.

### *3.3. Five-state lifecycle and idempotent execution*

Each rule passes through candidate, verified, active, degraded, and retired states under two signals. Confidence rises with accepted executions and falls with corrections; freshness decays exponentially with a seven-day base time constant and recovers when the rule fires again. A candidate rule needs three evidence samples and a Wilson confidence of at least 0.7 to become verified, and a verified rule whose confidence reaches 0.85 becomes active, the state in which it may act locally. Both thresholds sit on the same confidence scale as the calibration machinery: 0.85 matches the nominal conformal coverage target, so an active rule never promises more coverage than it can defend, and 0.7 requires the Wilson lower bound to clear chance agreement on the evidence collected so far. A rule whose freshness drops below 0.2 degrades, and a degraded rule that continues to drift retires. The executor adds hysteresis and an actuator mutex so a rule cannot oscillate an output faster than a cooldown window, and it records the resulting device state so re-deciding the same state is idempotent. The lifecycle matters for two reasons the ablations quantify later. It keeps stale rules from acting forever, and it gives the selector a signal about which rules are trustworthy at any moment. Figure 2 traces these states and the evidence and freshness signals that drive them.

### *3.4. PMS: perturbed-mean selection*

When several rules match one sensor state, the executor must pick one. Exact Thompson sampling stores float posterior parameters and samples Beta random variates, a path that is impractical on microcontroller-class hardware and that ordinal approximations on FPGA SoCs also seek to avoid [18]. PMS approximates the same exploration with two uint8 counters per rule. Alpha counts accepted executions and beta counts corrections; the score is alpha over

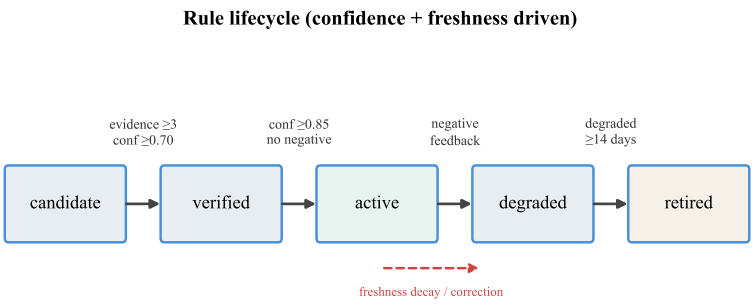



Figure 2: Five-state rule lifecycle: candidate → verified → active → degraded → retired, driven by Wilson confidence and exponentially decaying freshness.

alpha plus beta, perturbed by uniform noise from an xorshift32 generator with a magnitude of one over alpha plus beta plus one, and the highest perturbed score wins. This is a follow-the-perturbed-leader scheme in the sense of Kalai and Vempala [21], not Thompson sampling, and we do not claim its regret bound. Five hundred rules cost one kilobyte. The noise term keeps a high-confidence rule from permanently shadowing an under-explored one, which only matters when the best rule changes, so we report both stationary and non-stationary regret in Section 5.5.

### *3.5. MCU implementation*

The firmware targets an ESP32-S3 (N16R8, 512 KB SRAM, 8 MB PSRAM) under ESP-IDF 5.2.6 and FreeRTOS, with pinned task placement and static allocation for the confirmation worker. A rule is a cJSON object: conditions over sensor fields, an action, confidence, evidence, and freshness. Matching walks the condition list once, and conflict resolution applies the PMS score. The firmware ships with a 30-day default for the freshness time constant; all pipeline experiments reported here use the seven-day value. Rules persist to SPIFFS and are broadcast to a web dashboard over a WebSocket. Because

the board carries no sensor breakout, sensor readings are injected over USB serial from a replay of a dataset, and the same serial link carries a command-line interface for rule inspection. This trace-driven hardware-in-the-loop setup keeps the sensing path reproducible across experiments while exercising the real decision, execution, persistence, and logging paths on hardware. An on-device `esp_timer` wraps the match call so latency is measured, not simulated.

## 4. Experimental Setup

### *4.1. Datasets*

We instantiate the system in environment control, the canonical embedded-control task, and evaluate four synthetic datasets plus five real datasets that span three application domains. Four synthetic datasets generate 30-day streams of temperature, humidity, light, and motion with sinusoidal day and night structure, giving 465 to 549 interactions per run[2]. Each of the nine datasets is repeated four times with fixed seeds (42, 123, 999, 777), giving 36 held-out blocks. The interaction schedule is generated once, before the control loop starts, so a seed pins the sensor and user-input sequence independently of how the LLM behaves. That is what makes the cross-teacher comparison in Section 5.4 legitimate.

For real data, the UCI occupancy dataset provides physical readings (temperature, humidity, light, $CO_2$, and occupancy) recorded in an office [28]. Each run replays 600 chronologically ordered snapshots across 30 days, and because the teacher leaves most comfortable states untouched, autonomy is intrinsically capped by the data. We run six independent repeats, four fixed seeds plus two additional runs, and report the resulting variance. SML2010 contributes roughly 43 days (4,137 readings at 15-minute intervals) of indoor-climate readings (temperature, humidity, light, and $CO_2$) from a domotic house [33]. The Steel Industry Energy Consumption dataset gives a year of 15-minute industrial

[2]Counted from the released trace files.

readings (energy usage, reactive power, power factor, and $CO_2$) [34]. The Air Quality dataset gives over a year of hourly urban readings (CO, $NO_x$, $NO_2$, temperature, and humidity) with missing entries treated as absent fields [35]. Each run of these three datasets replays 600 chronologically ordered snapshots spread across the full span of the data, matching the UCI protocol. These three domains deliberately move beyond the smart-home vocabulary of fan, light, and curtain controls, so a method must read each domain's action vocabulary from its own traces. The CASAS Aruba-1 dataset in its STRANDS redistribution contributes 1,440 interactions per run with real activity and location annotations. Its numeric sensor fields are synthetic completions of the original PIR, door, and temperature sensors [29]. Its autonomy is high but its agreement with the teacher is zero, so we use STRANDS for coverage claims only and treat it as a deliberate failure sample for fidelity rather than a coverage showcase, a decision justified in Section 5.2.

### *4.2. Baselines*

Eight methods share the held-out evaluation, and two similarity caches are assessed separately in the supplementary analysis. Pure Cloud sends every interaction to the LLM: it defines the teacher's own behavior and is the floor on autonomy, since it never acts locally. Exact Cache replays a decision only when the exact sensor snapshot was seen before. The two caches are a sensor-vector cache over the four normalized sensor fields (16 bytes per entry, MCU-feasible) and a semantic cache over 384-dimensional MiniLM embeddings. An ESPHome-style fixed-threshold state machine is implemented in our codebase as a supplementary reference; it does not learn from LLM decisions and is omitted from the main comparison. User-defined Rules encode the ten hand-written automations a typical user would configure. LLM One-shot asks the LLM once for ten rules on the first day and freezes them, the WireClaw configuration pattern. Decision Tree trains a CART (depth 5, minimum leaf size 3) on all labeled decisions from the warm-up window and predicts with a probability threshold of 0.5. Decision Tree (online refit) applies the same learner but retrains daily on decisions ob-

served so far, matching the information regime of our system. ESP-Claw-style learns interval rules incrementally from cloud decisions as a simplified stand-in for a scripted, LLM-supervised engine. DistillToMCU runs COMIC and PMS end to end.

### *4.3. Evaluation protocol*

The protocol has two parts. First, a held-out time split: every method warms up or trains on days 1 to 21 and is evaluated only on days 22 to 30. Batch methods see the warm-up window as their training set; online methods (DistillToMCU, the online tree, ESP-Claw-style) learn from past decisions as the stream proceeds and are evaluated on the same window. Autonomy rate is the share of evaluation-window interactions handled locally, and cloud call reduction is its complement. This closes the training-on-test inflation that an in-sample replay would allow. Second, a teacher-replay pass measures what coverage is worth. For each dataset we sample 60 snapshots spread over the full horizon and ask the teacher LLM to decide three times at temperature zero, twice at temperature 0.7, and once for the cross-model teacher, using the same prompt, tools, and user input as the online runs. The majority of the three temperature-zero answers is the ground-truth action. Precision is the share of a method's local actions that match the teacher, recall is the share of teacher actions the method both covers and matches, and decision agreement is the same quantity as recall. The teacher's self-agreement baseline is the share of snapshots on which the three temperature-zero answers agree. Agreement carries a bootstrap 95% confidence interval over the 60 snapshots, and the precision/recall values in Table 2 carry Wilson binomial intervals, reported below the table. Because UCI is the lowest-action real dataset, its 60-snapshot cells rest on few teacher actions; we therefore add a 480-snapshot replay of UCI under three fixed seeds (1,440 snapshots) that stabilizes the precision comparison of Section 5.3; this is a stabilization choice for low-action data rather than a power analysis, and the 60-snapshot protocol remains the primary cell. Significance across methods uses Friedman tests with Nemenyi post-hoc comparisons [30]: 36 blocks by 8

methods for autonomy and 9 blocks by 8 methods for decision agreement. The 36 autonomy blocks are 16 synthetic, 4 STRANDS, 4 UCI, and 12 runs over SML2010, Steel, and Air Quality; two further UCI repeats inform the UCI variance estimate only.

### *4.4. Hardware and models*

The ESP32-S3 firmware matches rules, drives the on-board RGB actuator, persists the rule store, and logs traces. Sensor values arrive over USB serial from the dataset replay, which is the trace-driven HIL setting: the decision, execution, persistence, and logging paths are real hardware, while the sensing front end is replayed, following the hardware-feedback evaluation style of Embedded Arena [31]. An `esp_timer` measures the match call on the device. The teacher is DeepSeek V4 Flash for all online runs, with Qwen 3.7 Flash as the cross-model teacher. Across all protocols the study consumed 19,234 real cloud calls: 6,226 in the 36 online held-out blocks and two extra UCI variance repeats, 806 in same-data cross-teacher runs, 3,240 in teacher replay (nine datasets by 60 snapshots by six queries), 8,640 in the extended UCI replay (three seeds by 480 snapshots by six queries), and 322 in scripted cross-model pairing.

## 5. Results

### *5.1. Autonomy and cloud-call reduction*

Table 1 reports mean autonomy and cloud-call reduction (CCR, the share of evaluation-window interactions handled locally) over the 36 held-out blocks. The two decision-tree variants lead (99.7% and 99.4%), followed by hand-written rules (91.6%). The two 0.0 rows are by construction: Pure Cloud always consults the LLM, and Exact Cache never encounters an exactly repeated snapshot in the evaluation window, which is the point of including it. DistillToMCU reaches 85.4% with a standard deviation of 19.3, driven mostly by UCI, where the teacher rarely acts, and by the two most variable real domains, Steel and Air Quality. A Friedman test over the 36 blocks finds overall differences ($\chi^2 = 189.6$,

Table 1: Mean autonomy rate and cloud-call reduction over 36 held-out blocks (days 22 to 30), with standard deviation.

| Method | Autonomy (%) | CCR (%) |
|---|---|---|
| Pure Cloud | $0.0 \pm 0.0$ | $0.0 \pm 0.0$ |
| Exact Cache | $0.0 \pm 0.0$ | $0.0 \pm 0.0$ |
| User-defined Rules | $91.6 \pm 10.3$ | $91.6 \pm 10.3$ |
| LLM One-shot | $73.4 \pm 36.7$ | $73.4 \pm 36.7$ |
| Decision Tree (batch) | $99.4 \pm 1.5$ | $99.4 \pm 1.5$ |
| Decision Tree (online refit) | $99.7 \pm 1.6$ | $99.7 \pm 1.6$ |
| ESP-Claw-style | $75.8 \pm 21.0$ | $75.8 \pm 21.0$ |
| DistillToMCU | $85.4 \pm 19.3$ | $85.4 \pm 19.3$ |

$p < 0.001$). Nemenyi post-hoc comparisons place DistillToMCU statistically indistinguishable from hand-written rules (rank difference 0.03), one-shot rules (0.19), and ESP-Claw-style (1.22), and significantly below the decision-tree variants (1.86 and 2.14), a gap Section 6 returns to. One-shot's 73.4% mean hides a domain collapse: its frozen smart-home rules reach 0.0% autonomy on Steel and 15.0% on SML2010, while DistillToMCU reads each domain's action vocabulary from its traces and holds 69.7% and 79.3%. On UCI the six full-horizon repeats give 32.4% mean autonomy (individual repeats 9.2% to 55.0%) with a bootstrap 95% interval (percentile bootstrap, 10,000 resamples) of 18.2% to 46.5%, which we report as a property of low-action real data rather than a hidden failure. With $n = 6$ the percentile interval is wide, so the repeat range is reported alongside it. Table 1 reports the held-out window (days 22–30), where the four seeded UCI blocks average 48.2% (19.4% to 83.9%); the full-horizon figures above cover all 30 days including the low-autonomy warm-up, which is why the two UCI numbers differ. Figure 3 plots the learning curves behind these means, and Figure 4 visualizes the eight-method comparison of Table 1.

Similarity caches are excluded from the 36-block design. In the supplementary two-dataset sweep the 4-field sensor-vector cache reaches 71.2–99.4% auton-

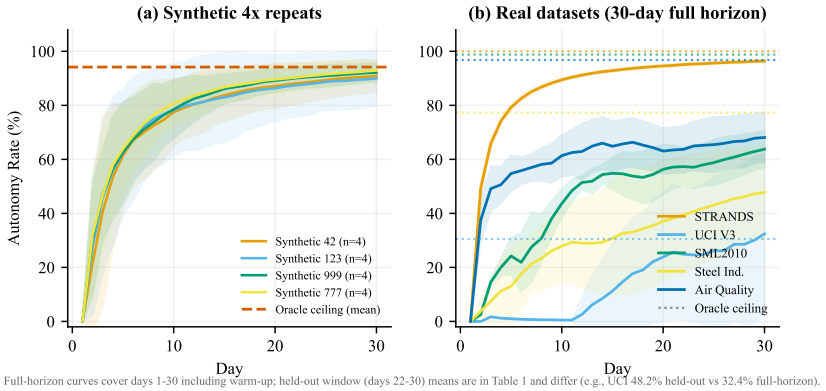



Figure 3: Daily autonomy learning curves over 30 days, per-dataset means across the four seeded runs (36 held-out blocks in total; full-horizon curves include the warm-up, so endpoints differ from the held-out means in Table 1).

omy and the 384-dimensional MiniLM semantic cache 0.9–97.2% across thresholds on seed42 and UCI.

### *5.2. Decision agreement and the teacher's self-agreement baseline*

Coverage only matters if local actions match the teacher. The teacher itself is imperfectly reproducible: across the nine datasets its three temperature-zero answers agree on 80.0% to 96.7% of snapshots. We use this self-agreement rate as the normalization baseline for fidelity, because a faithful student inherits the teacher's own noise. The teacher majority vote is the only oracle available in our setting: no independent record exists of what the device should have done, so fidelity is measured against that majority and normalized by the teacher's own reproducibility rather than claimed as absolute correctness; the 80.0% to 96.7% self-agreement range is the scale on which any student's fidelity must be read. DistillToMCU, restricted to rules distilled from the warm-up window so it faces the same information as the batch baselines, reaches 52.4%, 76.3%, 78.4%, and 54.3% agreement on the four synthetic datasets, that is 57.1% to 88.8% of the self-agreement baseline. On UCI it reaches 38.5% agreement (40.5% of the 95.0% baseline), on SML2010 87.5%, on Steel 79.2% (81.9% of the 96.7% base-

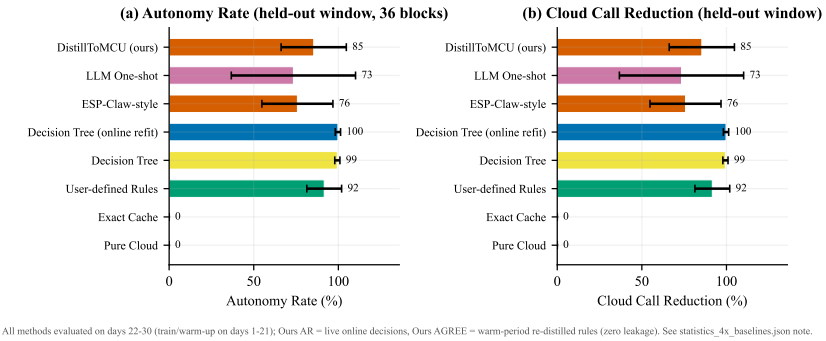



Figure 4: Mean held-out autonomy rate and cloud-call reduction over 36 blocks (days 22–30); Friedman/Nemenyi results are reported in Section 5.1.

line), and on Air Quality 35.2% (38.4% of the 91.7% baseline). On SML2010 the student's agreement exceeds the teacher's full-snapshot self-agreement (87.5% versus 80.0%) because the two denominators differ, so we never treat the ratio as a hard upper bound. On STRANDS the warm-up rules match every sampled snapshot but agree with none of the teacher's actions: the synthetic sensor completions leave the interval learner with overlapping, over-general conditions, and the final-day rule set improves to 46.5%. We therefore exclude STRANDS from fidelity claims and restrict it to coverage. The scoping decision is deliberate and explicit, and the traces released upon acceptance will let readers re-derive the 46.5% final-day figure. Across the nine blocks the Friedman test on agreement is significant ($\chi^2 = 45.1$, $p < 0.001$). Under Nemenyi (critical difference 2.47), DistillToMCU is not significantly different from either tree variant (rank difference 1.06), ESP-Claw-style (0.06), or hand-written rules (1.94), and is significantly above one-shot rules (2.78). The trees win the mean (online-refit 69.0%, batch 65.8%), and ESP-Claw-style also leads Ours on mean agreement (64.0% vs 55.8%), which is the expected price of their supervised batch training, and the precision column shows where that advantage leaks away.

*5.3. Precision and recall: the case for calibrated abstention*

Autonomy rewards acting; precision charges for acting wrong. Table 2 reports both on the teacher-replay snapshots, and the extended UCI replay below the table stabilizes the comparison against the small action counts of the 60-snapshot protocol. On UCI the batch tree acts on 100% of snapshots but agrees with the teacher on 11.7% of its actions (16.5% on the pooled 1,440-snapshot replay); the online tree is similar at 15.0% (18.3% pooled). DistillToMCU acts on 10 of the 60 snapshots and keeps 50.0% precision, and on the pooled replay holds $50.0 \pm 2.1\%$ precision at $46.5 \pm 2.9\%$ recall. ESP-Claw-style shows 100% precision on the 60-snapshot replay, a 10/10 small-sample artifact: over 1,440 snapshots its precision falls to $22.3 \pm 0.8\%$ at $85.8 \pm 2.7\%$ recall, so it sits on the recall side of the trade-off while DistillToMCU keeps the calibrated middle. Both frozen-rule families collapse on UCI: hand-written and one-shot rules fire on nearly every snapshot with 0.0% precision, and the exact cache holds $92.5 \pm 3.1\%$ precision at only 10% coverage. Read together, the UCI columns define DistillToMCU's operating point: meaningful coverage without reckless action.

This operating behavior is the selective-prediction pattern formalized by recent dual-threshold conformal prediction [36]: a conformal threshold guarantees the validity of a prediction set, and a separate abstention threshold controls selectivity by deciding when the system may decline to act. COMIC instantiates both signals—the 85% conformal coverage target fixes validity, and the lifecycle's confidence gates (0.7/0.85) act as the abstention threshold that decides when a match is trusted enough to execute locally rather than fall back to the cloud. On UCI this abstention threshold keeps the operating point at 50.0% precision and 20.8% local-action coverage, whereas the fully selective baselines trade the entire precision advantage for coverage. Figure 5 plots the same trade-off as coverage-risk operating points: DistillToMCU sits at low coverage with bounded risk, the exact cache is even more conservative, and the fully selective baselines buy full coverage at two to three times the risk.

The same advantage appears on the two other real domains whose action

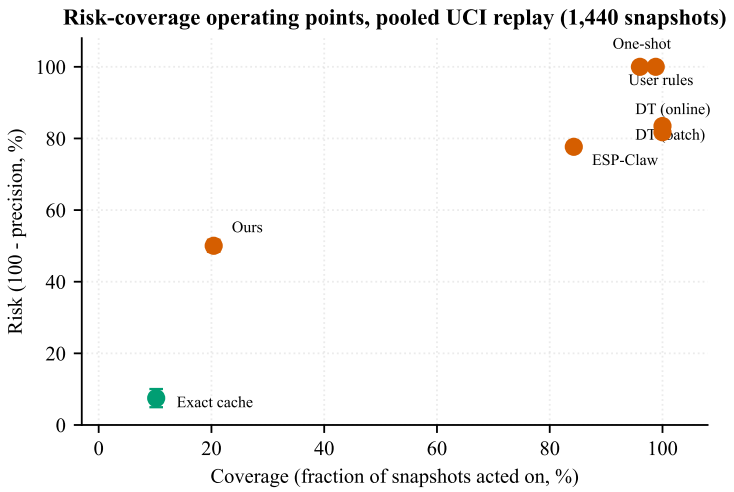



Figure 5: Coverage-risk operating points on the pooled UCI replay (1,440 snapshots, three seeds). Risk is 100 minus teacher-replay precision; coverage is the fraction of snapshots on which the method acts. Error bars are standard deviations across seeds.

vocabulary differs from smart-home defaults. On Steel, Ours keeps the highest precision (52.8% versus 30.0% for the batch tree, 28.3% for the online tree, and 47.4% for ESP-Claw-style), and on SML2010 it leads with 83.1% versus 70.0%, 70.0%, and 70.2%. Air Quality is the honest exception: with strongly coupled gas readings and overlapping intervals, Ours falls to 33.3% precision while the batch tree reaches 83.3% and ESP-Claw-style 97.8%. We report this as a failure mode rather than a footnote (Sections 6 and 7). All Ours cells use the warm-up rule set so that it faces the same information as the batch baselines; the online tree and ESP-Claw-style are evaluated on their full-stream state. The final day-30 rule set, which is what a deployed device would carry, reaches 7.1% precision ($n = 42$) on UCI and 35.0%, 35.0%, 25.9%, and 13.6% on the four synthetic datasets, because interval rules over-generalize as evidence accumulates without negative samples; both sets are reported in the Table 2 note. On the synthetic sets the teacher is dense in its actions and consistent enough (88.3–91.7% self-agreement) that the tree memorizes its warm-up labels, and its precision exceeds ours on three of the four synthetic datasets, so the synthetic

Table 2: Teacher-replay precision and recall (60 snapshots per dataset, teacher majority ground truth).

| Dataset | Ours P/R | Batch-DT P/R | Online-DT P/R | ESP-Claw P/R |
|---|---|---|---|---|
| Synthetic 42 | 36.7 / 52.4 | 43.3 / 61.9 | 52.5 / 73.8 | 30.0 / 35.7 |
| Synthetic 123 | 50.9 / 76.3 | 50.0 / 78.9 | 51.8 / 76.3 | 46.7 / 55.3 |
| Synthetic 999 | 48.3 / 78.4 | 64.9 / 64.9 | 43.3 / 70.3 | 41.2 / 56.8 |
| Synthetic 777 | 31.7 / 54.3 | 35.0 / 60.0 | 40.0 / 68.6 | 54.5 / 51.4 |
| STRANDS | 0.0 / 0.0 | 21.7 / 30.2 | 21.7 / 30.2 | 51.7 / 69.8 |
| UCI | 50.0 / 38.5 | 11.7 / 53.8 | 15.0 / 69.2 | 100.0 / 76.9 |
| SML2010 | 83.1 / 87.5 | 70.0 / 75.0 | 70.0 / 75.0 | 70.2 / 71.4 |
| Steel | 52.8 / 79.2 | 30.0 / 75.0 | 28.3 / 70.8 | 47.4 / 75.0 |
| Air Quality | 33.3 / 35.2 | 83.3 / 92.6 | 78.3 / 87.0 | 97.8 / 83.3 |

rows cannot separate calibration from memorization (Section 6). Averaged over the nine datasets, the batch tree's mean precision and recall are 45.5% and 65.8% against 43.0% and 55.8% for DistillToMCU, an edge driven by the synthetic and air-quality rows. Precision and recall here measure fidelity to the teacher majority, not user-validated correctness. Section 7 discusses that boundary. Figure 6 plots the precision–recall trade-off per dataset and the nine-dataset method means.

UCI 95% Wilson intervals (60-snapshot; $n_{\text{teacher\_act}} = 13$; $n_{\text{local}} = 10$ for Ours and ESP-Claw-style, 60 for each decision tree): Ours precision 23.7–76.3% (5/10), recall 17.7–64.5% (5/13); batch-DT precision 5.8–22.2% (7/60), recall 29.1–76.8% (7/13); online-DT precision 8.1–26.1% (9/60), recall 42.4–87.3% (9/13); ESP-Claw precision 72.2–100% (10/10), recall 49.7–91.8% (10/13). On the pooled 480-snapshot × 3-seed UCI replay (1,440 snapshots, 316 teacher actions): Ours $50.0 \pm 2.1\%$ precision and $46.5 \pm 2.9\%$ recall; batch-DT $16.5 \pm 0.5\%$ and $75.3 \pm 0.8\%$; online-DT $18.3 \pm 0.5\%$ and $83.2 \pm 1.3\%$; ESP-Claw-style $22.3 \pm 0.8\%$ and $85.8 \pm 2.7\%$; exact cache $92.5 \pm 3.1\%$ and $43.1 \pm 2.2\%$; hand-written and one-shot rules 0.0% each. Ours is evaluated on the warm-up rule set (final set: 7.1/23.1 on UCI; 35.0/50.0, 35.0/55.3, 25.9/40.5, 13.6/22.9 on the four synthetic datasets); the online tree and ESP-Claw-style use full-stream

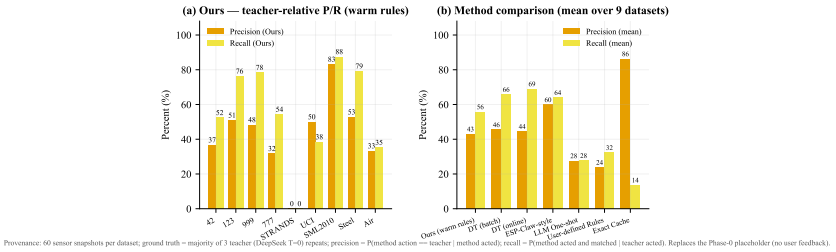


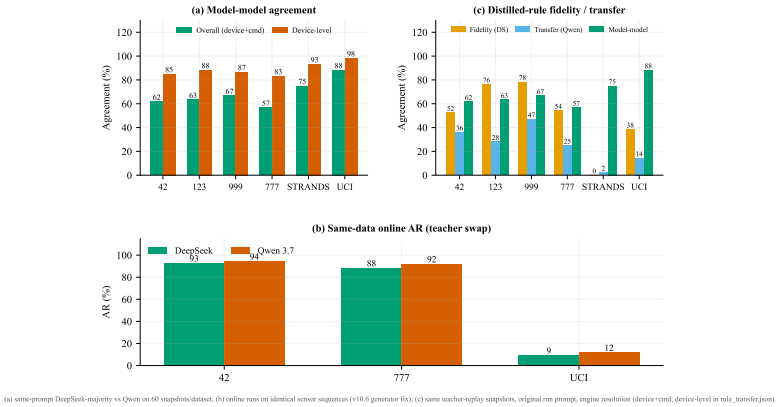

Figure 6: Precision–recall on the 60-snapshot teacher replay: (a) per dataset with the warm-up rule set, (b) method means over nine datasets.

state.

### *5.4. Cross-teacher transfer*

Three measurements bound how tightly the system is tied to one model. First, the same-data online runs: with identical sensor sequences, swapping DeepSeek for Qwen moves final autonomy by 1.7 percentage points on the first synthetic dataset (92.7% to 94.4%), 3.7 percentage points on the seed777 dataset (88.3% to 92.0%), and 2.6 percentage points on UCI (9.2% to 11.8%)[3]. Second, model-to-model agreement under the deployed prompt is 56.7% to 88.3% at the device-plus-command level and 83.3% to 98.3% at the device level, clearly lower than the 94.0–96.2% agreement the earlier paired experiments yield under a scripted rule prompt across the three paired datasets (seed42, seed777, and UCI), which indicates that prompt and scenario, not the models, dominate the gap. Third, DeepSeek-distilled rules transfer to Qwen decisions at 25.0% to 47.1% on synthetic data while their fidelity to DeepSeek itself is 52.4% to 78.4%, so the rules retain a meaningful share of the inter-model distance. The system is teacher-agnostic in the practical sense: the teacher can be replaced, and the distilled rule set remains a faithful student of whichever model taught it.

[3]The synthetic runs share a seed, so their sensor and user-input sequences are identical; for UCI the sensor sequences are identical across the two backends while the older Qwen run used slightly different query formatting.



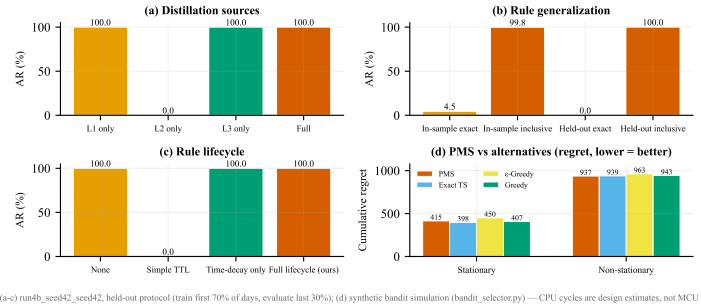

(a) same-prompt DeepSeek-majority vs Qwen on 60 snapshots/dataset; (b) online runs on identical sensor sequences (v10.6 generator fix); (c) same teacher-replay snapshots, original run prompt, engine resolution (device+cmd; device-level in rule_transfer.json).

Figure 7: Cross-teacher results: same-data autonomy deltas on three paired datasets (42, 777, UCI), and same-prompt model agreement plus distilled-rule fidelity/transfer on the six datasets with dual-model runs.

Figure 7 summarizes the same-data deltas, model agreement, and rule-transfer measurements.

### *5.5. Ablations*

Four ablations isolate the components, all on the first synthetic dataset under the held-out protocol. Their agreement cells count two replay snapshots ($n = 2$) and are therefore omitted from the main text. The distillation-source ablation shows that L1 (tool-call intervals) alone yields four rules and 100% held-out autonomy, L3 (sensor-action correlation) yields three rules and 100%, and the combination yields five rules and 100%, while L2 alone yields no rules and 0%. The generalization ablation contrasts inclusive intervals with exact matching: inclusive intervals hold 100% held-out autonomy where exact matching falls to 0%, a gain of 100 percentage points a held-out gain of 100 percentage points versus 95.3 points in-sample, so most of the value is genuine generalization rather than memorization. The lifecycle ablation is the sharpest: a fixed seven-day TTL expires every rule and collapses autonomy to 0% (the intended finding), while the full freshness-and-evidence lifecycle holds 100%. For selection, exact Thompson sampling attains 397.5, greedy 407.0, and PMS 414.6 in a




Provenance: (a-c) run4b_seed42_seed42, held-out protocol (train first 70% of days, evaluate last 30%); (d) synthetic bandit simulation (bandit_selector.py) — CPU cycles are design estimates, not MCU measurements.

Figure 8: Ablation results (single dataset, held-out; agreement cells $n = 2$).

stationary bandit environment, so the FTPL approximation costs 17.1 against the exact posterior and 7.6 against greedy when the best arm never changes; in a non-stationary environment with drift every 500 rounds, PMS attains 937.0 against 943.3 for greedy, 962.8 for epsilon-greedy, and 939.1 for exact Thompson sampling, at two bytes per rule instead of eight. For flash policy, batching trace writes cuts writes from 17.0 to 1.1 per day without changing endurance materially, so the claim is write-amplification reduction, not lifetime extension. Figure 8 reports all four ablations.

### *5.6. Hardware*

On-board rule matching, including the JSON condition walk, completes in 1.48 ms at p50 in three independent sessions of 100 matches each (mean 1.53 to 1.60 ms, p95 2.19 ms, p99 2.19 to 2.29 ms, with one 8.97 ms outlier in a single session), three orders of magnitude below the measured cloud round trip. The device sustains 200/200 injected sensor events without loss, holds final autonomy of 94% to 99% in the three 100-match latency sessions and 98% to 100% in the 200-event injection runs (18% to 23% on UCI: 18% on the DeepSeek runs, 23% on the Qwen run), with 173 to 178 KiB of SRAM free and 7.98 MiB of PSRAM available. Table 3 collects these on-device measurements. Figure 9 contrasts the on-device match latency with the measured cloud round trip.

Table 3: On-device measurements on the ESP32-S3. Latency statistics come from three independent sessions of 100 rule matches each; execution figures come from 200-event injection runs. Free SRAM values refer to those injection runs; in the three 100-match latency sessions, free SRAM was 165.8–175.4 KiB depending on the retained rule set and state.

| Metric | Value |
|---|---|
| p50 match latency | 1.48 ms (1.475–1.476 across sessions) |
| Mean match latency | 1.53–1.60 ms |
| p95 / p99 match latency | 2.19 ms / 2.19–2.29 ms |
| Injected events lost | 0 of 200 (all runs) |
| Synthetic autonomy, 100-match sessions | 94–99% |
| Synthetic autonomy, 200-event runs | 98–100% |
| UCI autonomy | 18–23% |
| Free SRAM after run | 173–178 KiB (of 512 KiB) |
| Free PSRAM after run | 7.98 MiB (of 8 MiB) |
| Distilled rules on board | 4–8 |

## 6. Discussion

The results support three claims and force one concession. The concession is autonomy: on predictable data a supervised decision tree covers more interactions, and the paper does not claim otherwise. The claims follow from reading coverage together with the teacher-replay columns. First, precision and recall are the honest pair of metrics for this task, because a method that acts on everything is trivially autonomous but measurably wrong on real data. The UCI gap between the tree's 16.5% precision and our warm-up set's $50.0 \pm 2.1\%$ on the pooled replay is the quantitative version of that sentence. Second, fidelity must be read against the teacher's self-agreement: the teacher disagrees with itself on up to 20.0% of snapshots, so the majority-vote target inherits that ambiguity, and reporting our fidelity as 57% to 89% of the self-agreement baseline on synthetic data (38.4% to 81.9% on the three informative real datasets) separates student error from teacher noise. Third, the distilled rules are a property of the behavior stream rather than of one model, which the 1.7 to 3.7 percentage point teacher-swap deltas and the nonzero cross-model transfer sup-

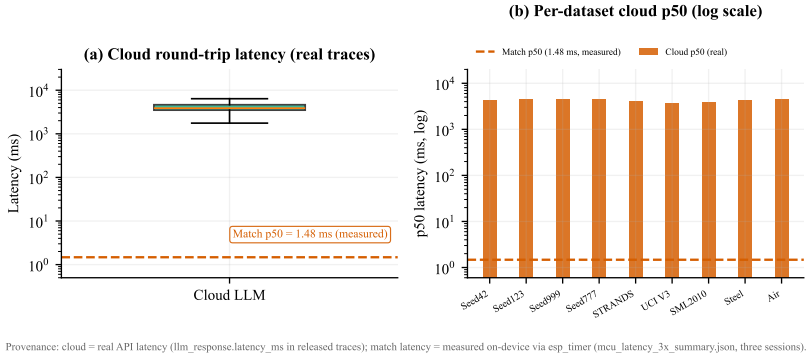



Figure 9: Measured on-device match latency versus cloud round trip.

port. Against the closest stand-in for a scripted, LLM-supervised incremental engine (ESP-Claw-style), the calibrated pipeline keeps higher precision on the same task: 50.0% versus 22.3% on the pooled UCI replay, 52.8% versus 47.4% on Steel, and 83.1% versus 70.2% on SML2010, at the cost of recall; Air Quality is the exception (33.3% versus 97.8%).

Synthetic data favors the tree for a simpler reason: the teacher acts densely and consistently enough (88.3–91.7% self-agreement) for a supervised tree to memorize the warm-up window, so its precision exceeds ours on three of the four synthetic datasets. That regime cannot distinguish calibration from memorization. UCI is the opposite regime: the teacher acts sparsely and stochastically, aggressive coverage burns precision, and the abstention advantage appears. Since low-action, occasionally-changing real environments are exactly where a deployed device cannot afford wrong actuations, we argue UCI is the deployment-relevant test and report both regimes rather than selecting either. Part of the tree's low precision may also reflect the fixed CART configuration (depth 5, threshold 0.5) rather than aggressive coverage alone; we hold one learner fixed across datasets for comparability.

Two domain-level findings sharpen the picture. First, frozen rules do not transfer across domains: the one-shot smart-home rules hold 0.0% autonomy on Steel and 15.0% on SML2010, and both frozen-rule families act on nearly

every UCI snapshot with 0.0% precision, whereas DistillToMCU reads each domain's action vocabulary from its own traces (69.7% and 79.3% autonomy; 52.8% and 83.1% precision). Second, Air Quality marks the boundary of the interval learner: correlated gas channels produce overlapping conditions whose precision falls to 33.3%, while the tree (83.3%) and ESP-Claw-style (97.8%) win. Coupled-sensor domains therefore remain open, and Section 7 records this as a limitation rather than an edge case.

Cross-model agreement also depends on the prompt. On the first synthetic dataset (seed42) the scripted-rule-prompt pairing reaches 96.2% agreement while the deployed-prompt replay reaches 61.7%, a 34.5-point gap, because the scripted prompt hands both models the same explicit thresholds while the deployed prompt leaves behavior implicit. This is a warning for the field's standard practice of reporting cross-model consistency under simplified prompts, and a reason our transfer measurements all run under the deployed prompt.

## 7. Limitations

Four boundaries qualify the results. Evaluation is hardware-in-the-loop rather than a field deployment: sensing is replayed over USB, and precision, recall, and agreement are judged against the teacher LLM, so they measure fidelity rather than correctness. STRANDS uses synthetic sensor completions and is therefore limited to coverage claims; the bootstrap 95% interval for UCI autonomy spans 18.2% to 46.5% (individual repeats 9.2% to 55.0%). Power and flash-endurance figures are analytical rather than measured (the power estimate uses the ESP32-S3 datasheet's typical active-mode current), the ablations run on one dataset with two agreement snapshots, and on-device evaluation covers fixed-length injection sessions rather than days-long soak testing. The interval learner's limit appears on Air Quality, where correlated gas channels produce overlapping conditions and precision falls to 33.3% against 83.3% for the tree. The mechanism is structural: the five gas channels (CO, $NO_x$, $NO_2$, temperature, humidity) co-vary in urban pollution episodes, so axis-aligned in-

tervals cannot separate them—a rule conditioned on CO alone also fires during $NO_x$ episodes where the teacher abstains, and the overlapping intervals defeat the discriminative-condition filter. This is a boundary of interval-shaped rules rather than a tuning artifact, and it motivates future work on lightweight cross-sensor features an MCU can still evaluate in constant memory. When a rule has no selection counters, conflict resolution falls back to deterministic specificity (condition count) plus confidence, and coverage-weighted specificity is future work. Intervals also over-generalize as evidence accumulates: the final-day rule set reaches 7.1% precision on UCI (Table 2 note), and online pruning of the growing rule store is future work.

## 8. Conclusion

This paper shows that a microcontroller can inherit a cloud LLM's control behavior without keeping the LLM. COMIC distills structured tool-call decisions into confidence-calibrated interval rules with a five-state lifecycle, PMS selects among them with two bytes per rule, and the resulting system executes on an ESP32-S3 at 1.48 ms with no LLM at runtime. Across more than 19,000 real cloud calls spanning online runs, replay, and cross-model experiments, the system reaches 85.4% held-out autonomy over 36 blocks, statistically on par with hand-written, one-shot, and ESP-Claw-style rules, reads fidelity against the teacher's self-agreement baseline, keeps 50.0% precision with its warm-up rule set on real sensor data where a batch decision tree keeps 16.5% on the extended UCI replay, and survives teacher substitution with 1.7 to 3.7 percentage point shifts. Future work includes coverage-weighted conflict resolution, online rule pruning for long-horizon growth, coupled-sensor domain handling, long-term physical sensor deployments, and multi-device coordination.

## Data Availability Statement

The synthetic datasets, experiment scripts, firmware, and all result files used in this study are available in the public repository `https://github`

.com/Yunan0718/DistillToMCU. The real datasets are public: UCI Occupancy Detection (CC BY 4.0), SML2010 (UCI Machine Learning Repository, DOI 10.24432/C5RS3S, CC BY 4.0), Steel Industry Energy Consumption (UCI 851, DOI 10.24432/C52G8C, CC BY 4.0), and Air Quality (UCI 360, DOI 10.24432/C59K5F, CC BY 4.0). The CASAS Aruba-1 data is used in the redistribution published by the STRANDS project; the original CASAS recordings are openly available on Zenodo [32].

**Supplementary Material**

The Nemenyi critical-difference diagram for the held-out autonomy comparison, the online-versus-oracle capacity analysis, and the rule-store growth curves are provided as supplementary figures.

**Use of AI-Assisted Technologies**

During the preparation of this work, the author used AI-assisted tools – including large-language-model-based writing and editing assistants (OpenAI Codex and Jenni.ai) for drafting, revising, and language review – and used the DeepSeek and Qwen large language models as research subjects in the experiments described in Sections 4–5. All figures were generated programmatically from the experimental data with custom plotting scripts; no generative image tools were used. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

**CRediT Authorship Contribution Statement**

Yuxiang Zhang: Conceptualization; Methodology; Software; Validation; Formal analysis; Investigation; Data curation; Writing – original draft; Writing – review and editing; Visualization; Project administration.

## Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.


## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.


## References


[1] Z. Zhan, K. Li, Y. Zhang, and H. Haddadi, "HearthNet: Edge multi-agent orchestration for smart homes," in *Proc. ACM Conf. AI Agentic Syst. (CAIS)*, Demo Track, 2026, doi: 10.1145/3786335.3813188.

[2] J. Xiong and J. Bao, "EdgeTalk-MCU: State-aware prompt-constrained local LLM control with runtime shielding for low-latency microcontroller interaction," *Appl. Sci.*, vol. 16, no. 12, 2026, doi: 10.3390/app16125748.

[3] Espressif Systems, "ESP-Claw," GitHub repository, 2026. [Online]. Available: `https://github.com/espressif/esp-claw`, accessed Aug. 19, 2026.

[4] D. Yang, "Device Context Protocol: A compact, safety-first architecture for LLM-driven control of constrained devices," arXiv:2605.26159, 2026.

[5] B. Yuan, R. Song, Y. Su, S. Yang, and J. Qin, "ClawTrace: Cost-aware tracing for LLM agent skill distillation," arXiv:2604.23853, 2026.

[6] X. Gao, Y. Yao, Q. Zhang, K. Dong, A. Baidya, R. Guo, H. Hasson, and K. Das, "RIMRULE: Improving tool-using language agents via MDL-guided rule learning," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, San Diego, CA, USA, 2026, pp. 34631–34646, doi: 10.18653/v1/2026.acl-long.1599.

[7] Z. Guo, D. Qi, H. Gu, P. Cheng, and Y. Xiong, "SKILL-DISCO: Distilling and compiling agent traces into reusable procedural skills," arXiv:2606.26669, 2026.

[8] G. Li, R. Wu, and H. Tan, "A plan reuse mechanism for LLM-driven agent," arXiv:2512.21309, 2025.

[9] X. Li, Y. Zang, Y. Cao, and A. Sun, "Skill-as-Pseudocode: Refactoring skill libraries to pseudocode for LLM agents," arXiv:2605.27955, 2026.

[10] Y. Ouyang, Y. Xiao, Y. Gu, and X. Zhang, "SkCC: Portable and secure skill compilation for cross-framework LLM agents," arXiv:2605.03353, 2026.

[11] X. Zhang, G. Wang, Y. Cui, W. Qiu, Z. Li, B. Zhu, and P. He, "Experience Compression Spectrum: Unifying memory, skills, and rules in LLM agents," arXiv:2604.15877, 2026.

[12] Open-source project (no listed authors), "WireClaw: ESP32 AI agent with persistent memory and offline rule engine," GitHub repository, 2026. [Online]. Available: `https://github.com/M64GitHub/WireClaw`, accessed Aug. 19, 2026.

[13] D. Jeong and H. Woo, "On-device intent reasoning for smart home agents via ontology-augmented sLLMs," *IEEE Access*, vol. 13, pp. 197645–197662, 2025, doi: 10.1109/ACCESS.2025.3634621.

[14] H. Wang, G. Wang, H. Xiao, et al., "Skill-SD: Skill-conditioned self-distillation for multi-turn LLM agents," arXiv:2604.10674, 2026.

[15] Z. Yuan, J. Pan, X. Zhang, and D. Chen, "HomeSGN: A smarter home with novel rule mining enabled by a scorer-generator GAN," in *Proc. Asia South Pacific Design Autom. Conf. (ASP-DAC)*, 2024, pp. 102–108, doi: 10.1109/ASP-DAC58780.2024.10473909.

[16] Y. Hoffner, E. Kaufman, A. Amir, E. Yovel, and F. Harel, "Automation of smart homes with multiple rule sources," in *Proc. Int. Conf.*

*Internet Things, Big Data Security (IoTBDS)*, 2024, pp. 40–52, doi: 10.5220/0012556300003705.

[17] S. Agrawal and N. Goyal, "Near-optimal regret bounds for Thompson sampling," *J. ACM*, vol. 64, no. 5, Art. no. 30, 2017, doi: 10.1145/3088510.

[18] S. V. S. Santosh and S. J. Darak, "Multi-armed bandit algorithms on system-on-chip: Go frequentist or Bayesian?" arXiv:2106.02855, 2021.

[19] O. A. Hanna, L. Yang, and C. Fragouli, "Solving multi-arm bandit using a few bits of communication," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, PMLR, vol. 151, 2022, pp. 11215–11236. [Online]. Available: `https://proceedings.mlr.press/v151/hanna22a.html`, accessed Aug. 19, 2026.

[20] K. Wang, "MINTS: Minimalist Thompson sampling," arXiv:2606.01655, 2026.

[21] A. Kalai and S. Vempala, "Efficient algorithms for online decision problems," *J. Comput. Syst. Sci.*, vol. 71, no. 3, pp. 291–307, 2005, doi: 10.1016/j.jcss.2004.10.016.

[22] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.

[23] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Found. Trends Mach. Learn.*, vol. 16, no. 4, pp. 494–591, 2023, doi: 10.1561/2200000101.

[24] M. Greenwald and S. Khanna, "Space-efficient online computation of quantile summaries," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2001, pp. 58–66, doi: 10.1145/376284.375670.

[25] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962, doi: 10.1080/00401706.1962.10490022.

[26] P. D. Grünwald, *The Minimum Description Length Principle*. Cambridge, MA, USA: MIT Press, 2007.

[27] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.

[28] L. M. Candanedo and V. Feldheim, "Accurate occupancy detection of an office room from light, temperature, humidity and $CO_2$ measurements using statistical learning models," *Energy Buildings*, vol. 112, pp. 28–39, 2016, doi: 10.1016/j.enbuild.2015.11.071.

[29] D. J. Cook, A. S. Crandall, B. L. Thomas, and N. C. Krishnan, "CASAS: A smart home in a box," *Computer*, vol. 46, no. 7, pp. 62–69, 2013, doi: 10.1109/MC.2012.328.

[30] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006. [Online]. Available: `https://jmlr.org/papers/v7/demsar06a.html`, accessed Aug. 19, 2026.

[31] Z. Zhang, A. Le Metzger, J. Lyu, C.-C. Chang, et al., "Embedded Arena: Iterative optimization via hardware feedback," arXiv:2606.16190, 2026.

[32] D. J. Cook, "CASAS Smart Home dataset (aruba, cairo, milan, tulum) – free living, motion, door, temperature, activity labels," Zenodo, 2025, doi: 10.5281/zenodo.17180309.

[33] F. Zamora-Martínez, P. Romeu, P. Botella-Rocamora, and J. Pardo, "On-line learning of indoor temperature forecasting models towards energy efficiency," *Energy Buildings*, vol. 83, pp. 162–172, 2014, doi: 10.1016/j.enbuild.2014.04.034.

[34] V. E. Sathishkumar, C. Shin, and Y. Cho, "Efficient energy consumption prediction model for a data analytic-enabled industry building in a smart city," *Building Res. Inf.*, vol. 49, no. 1, pp. 127–143, 2021, doi: 10.1080/09613218.2020.1809983.

[35] S. De Vito, E. Massera, M. Piga, L. Martinotto, and G. Di Francia, "On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario," *Sensors Actuators B, Chem.*, vol. 129, no. 2, pp. 750–757, 2008, doi: 10.1016/j.snb.2007.09.060.

[36] S. Tayebati and A. R. Trivedi, "Beyond confidence: Adaptive abstention in dual-threshold conformal prediction for autonomous system perception," arXiv:2502.07255, 2025.

[37] S. Zhou, et al., "WALL-E 2.0: World alignment by neurosymbolic learning improves world model-based LLM agents," arXiv:2504.15785, 2025.

[38] J. Qiu, et al., "AgentDistill: Training-free agent distillation with generalizable MCP boxes," arXiv:2506.14728, 2025.